

Introduction to Gen AI

Nagesh

May 2026

”Ohh sorry !! I dont have any subjective judgements nor I offer any objective inferences ”
(Nagesh)

Contents

Foundational Problem of Generative AI	3
How Do We Solve It?	3
Important Notes and Mathematical Foundations	4
Example Workflow of Generative AI	7
Open Questions in Generative AI	10
Variational Divergence Minimization (VDM)	12
Properties of f -Divergence	12
Examples of f -Divergences	13
Key Insight	14
f -Divergence Minimization	15
Key Idea	15
Example	16
Law of the Unconscious Statistician (LOTUS)	16
Law of Large Numbers (LLN)	16
Interpretation	17
Why Large Datasets Matter	17

Core Idea Behind f -Divergence Minimization	17
Expressing f -Divergence in Terms of Expectations	18
Convex Conjugate Function	18
Aside : What is a Convex Function?	19
Properties of the Conjugate Function	20
Applying Conjugates to f -Divergence	21
Introducing a Function $T(x)$	21
Final Variational Form	22
Why is this Important?	22

Foundational Problem of Generative AI

Given a dataset

$$D = \{x_1, x_2, x_3, \dots, x_n\},$$

where

$$x_i \stackrel{\text{iid}}{\sim} P_X,$$

and P_X is an unknown data distribution.

The goal of Generative AI is to:

Estimate P_X and learn to sample from it.

That is, we seek a model capable of generating new samples

$$x \sim P_X$$

that resemble the observed data.

How Do We Solve It?

i) Assume a Parametric Family

We assume that the unknown distribution P_X belongs to a parametric family

$$P_\theta,$$

where θ denotes learnable parameters.

Typically,

$$P_\theta$$

is represented using a model, most commonly a deep neural network.

A model is therefore a parametric function used to approximate the unknown distribution P_X .

$$P_\theta \approx P_X$$

Examples include:

- Variational Autoencoders (VAEs)
- Generative Adversarial Networks (GANs)
- Diffusion Models
- Autoregressive Transformers

ii) Define a Divergence Metric

We define a divergence (or distance) measure between the true distribution P_X and the model distribution P_θ .

Common choices include:

$$D(P_X \parallel P_\theta)$$

Examples:

- Kullback–Leibler (KL) Divergence
- Jensen–Shannon Divergence
- Wasserstein Distance
- Maximum Mean Discrepancy (MMD)

This divergence quantifies how different the generated distribution is from the real data distribution.

iii) Solve an Optimization Problem

Finally, we optimize the model parameters θ such that the divergence is minimized:

$$\theta^* = \arg \min_{\theta} D(P_X \parallel P_\theta)$$

After optimization,

$$P_{\theta^*} \approx P_X,$$

allowing us to generate realistic samples from the learned distribution.

Important Notes and Mathematical Foundations

1. Starting Point of Generative AI

Our starting point is a dataset

$$D = \{x_1, x_2, x_3, \dots, x_n\}$$

where each sample is assumed to be independently and identically distributed (IID) from an unknown probability distribution P_X .

Mathematically,

$$x_i \stackrel{\text{iid}}{\sim} P_X$$

Each data sample belongs to a d -dimensional space:

$$x_i \in \mathbb{R}^d$$

Thus,

$$D \subset \mathbb{R}^d$$

Example: Images

Suppose our data consists of images.

An image can be represented as:

$$x_i \in \mathbb{R}^{C \times H \times W}$$

where:

- C = Number of channels
- H = Height of image
- W = Width of image

For example:

$$x_i \in \mathbb{R}^{3 \times 256 \times 256}$$

represents a colored RGB image of size 256×256 .

Flattening the image into a vector gives:

$$x_i \in \mathbb{R}^d$$

where

$$d = C \times H \times W$$

For the above example:

$$d = 3 \times 256 \times 256$$

2. Understanding the Random Variable X

Each data sample is an observation of a random variable X .

$$X \sim P_X$$

Suppose:

$$D = \{x_1, x_2, x_3, \dots, x_{1000}\}$$

and each x_i is an image.

Then we have 1000 image samples drawn from the same underlying distribution P_X .

Although:

$$x_1 \neq x_2$$

their pixel values may differ significantly, both are generated from the same distribution:

$$x_1 \sim P_X$$

$$x_2 \sim P_X$$

This means:

- Samples are statistically different
- But originate from the same probability law

3. Important Notation

Independence

$$X_i \perp X_j$$

means:

X_i and X_j are statistically independent

Sampling from Distribution

$$X_i \sim P_X$$

means:

X_i is sampled from distribution P_X

Thus:

$$x_1, x_2, x_3, \dots, x_n \stackrel{\text{iid}}{\sim} P_X$$

means:

- All samples come from the same distribution
- All samples are mutually independent

4. Why Do We Assume IID?

The IID assumption is mainly made for mathematical tractability.

Suppose the dataset is generated from multiple unknown distributions:

$$P_X^{(1)}, P_X^{(2)}, P_X^{(3)}, \dots$$

Then the learning problem becomes:

- Estimate multiple unknown distributions simultaneously
- Identify which sample belongs to which distribution
- Learn transitions and interactions between distributions

This is substantially more difficult.

Therefore, for simplicity and mathematical convenience, we assume:

$$x_i \stackrel{\text{iid}}{\sim} P_X$$

which allows us to estimate a single underlying distribution.

5. Vector Representation of Data

Each sample X_i is a vector-valued random variable of dimension d .

$$X_i \in \mathbb{R}^d$$

Explicitly:

$$X_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ x_{i3} \\ \vdots \\ x_{id} \end{bmatrix}$$

where:

d = number of features

Examples:

- Pixels in an image
- Audio amplitudes
- Word embedding dimensions
- Sensor measurements

Example Workflow of Generative AI

Suppose there exists a latent random variable

$$Z \in \mathbb{R}^k$$

with a simple and known probability distribution.

A common choice is:

$$Z \sim \mathcal{N}(0, I)$$

where:

- 0 is the zero mean vector
- I is the identity covariance matrix

Thus, Z is easy to sample from.

Generator Function

Now define a parametric function

$$g_\theta : \mathcal{Z} \rightarrow \mathcal{X}$$

that maps latent variables to the data space.

$$\tilde{X} = g_\theta(Z)$$

Here:

- Z is the latent random variable
- g_θ is typically a deep neural network
- $\tilde{X}$ is the generated sample

$$Z \sim \mathcal{N}(0, I)$$

$$Z \xrightarrow{g_\theta(\cdot)} \tilde{X} \sim P_\theta$$

The transformation g_θ changes the distribution of the random variable.
Although:

$$Z \sim \mathcal{N}(0, I)$$

the transformed variable:

$$\tilde{X} = g_\theta(Z)$$

generally follows a completely different distribution.
We denote the distribution of generated samples as:

$$P_\theta(\tilde{X})$$

or equivalently its density as:

$$p_\theta(\tilde{x})$$

Goal of Learning

Let:

$$P_X$$

be the true unknown data distribution.

We want the generated distribution:

$$P_\theta$$

to become as close as possible to:

$$P_X$$

To measure this closeness, define a divergence metric:

$$D(P_X \parallel P_\theta)$$

where:

- $D \geq 0$

-

$$D(P_X \parallel P_\theta) = 0 \iff P_X = P_\theta$$

Optimization Problem

The learning problem becomes:

$$\theta^* = \arg \min_{\theta} D(P_X \parallel P_\theta)$$

Thus, we optimize the parameters θ such that the generated distribution approximates the true data distribution.

What Happens After Optimization?

After solving the optimization problem:

$$P_{\theta^*}(\tilde{X}) \approx P_X(X)$$

This means:

- The neural network g_{θ^*} implicitly learns the data distribution
- Sampling from Z and passing it through g_{θ^*} generates realistic samples

$$Z \sim \mathcal{N}(0, I)$$

$$\tilde{X} = g_{\theta^*}(Z) \sim P_{\theta^*} \approx P_X$$

Hence, we can approximately sample from the unknown data distribution P_X .

Intuition

The neural network learns a transformation:

$$g_\theta : \text{Simple Distribution} \rightarrow \text{Complex Data Distribution}$$

That is:

$$\mathcal{N}(0, I) \longrightarrow P_X$$

This is one of the central ideas in modern Generative AI.

Open Questions in Generative AI

1. How Do We Compute Divergence Metrics?

The true distribution P_X is unknown.

Also, in many models:

$$p_\theta(x)$$

is not explicitly computable.

Therefore:

$$D(P_X \parallel P_\theta)$$

cannot always be computed directly.

Different generative models solve this differently:

- GANs use adversarial learning
- VAEs use variational lower bounds
- Diffusion models use denoising objectives
- Flow models use exact likelihoods

2. What Divergence Metric Should We Choose?

Different divergence metrics produce different behaviors.

Examples:

- KL Divergence
- Reverse KL Divergence
- Jensen–Shannon Divergence
- Wasserstein Distance
- Score Matching Objectives

The choice affects:

- Stability of training
- Quality of generated samples
- Diversity of samples
- Optimization difficulty

3. How Should We Choose g_θ ?

The architecture of:

$$g_\theta$$

determines the family of distributions:

$$P_\theta$$

Possible choices:

- CNNs for images
- Transformers for sequences
- U-Nets for diffusion models
- Autoregressive architectures

Thus, model design strongly influences:

- Expressivity
- Efficiency
- Scalability
- Sample quality

4. How Do We Solve the Optimization Problem?

The optimization problem is highly non-convex:

$$\theta^* = \arg \min_{\theta} D(P_X \parallel P_\theta)$$

Challenges include:

- Vanishing gradients
- Instability
- Mode collapse
- Large computational cost

Modern approaches use:

- Stochastic Gradient Descent (SGD)
- Adam optimizer
- Backpropagation
- Large-scale GPU training

Variational Divergence Minimization (VDM)

A central idea in Generative AI is to measure the difference between two probability distributions.

One very general family of divergence measures is called the ***f*-divergence** family.

***f*-Divergence Definition**

Given two probability distributions:

$$P_X \quad \text{and} \quad P_\theta$$

with corresponding density functions:

$$p_X(x) \quad \text{and} \quad p_\theta(x),$$

the *f*-divergence between them is defined as:

$$D_f(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_\theta(x) f\left(\frac{p_X(x)}{p_\theta(x)}\right) dx$$

where:

•

$$f : \mathbb{R}^+ \rightarrow \mathbb{R}$$

is a convex and lower semi-continuous function

•

$$f(1) = 0$$

•

$$\mathcal{X}$$

is the space on which both distributions are supported

Important Note:

The quantity

$$\frac{p_X(x)}{p_\theta(x)}$$

is a scalar-valued ratio between density functions.

Properties of *f*-Divergence

i) Non-Negativity

For every valid convex function *f*,

$$D_f(P_X \parallel P_\theta) \geq 0$$

ii) Identity Property

$$D_f(P_X \parallel P_\theta) = 0 \iff P_X = P_\theta$$

Thus, the divergence becomes zero only when the two distributions are identical.

Examples of f -Divergences

a) Kullback–Leibler (KL) Divergence

Choose:

$$f(u) = u \log u$$

Substituting into the f -divergence definition:

$$D_f(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_\theta(x) f\left(\frac{p_X(x)}{p_\theta(x)}\right) dx$$

gives:

$$D_{KL}(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_\theta(x) \left[\frac{p_X(x)}{p_\theta(x)} \log \left(\frac{p_X(x)}{p_\theta(x)} \right) \right] dx$$

Simplifying:

$$D_{KL}(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_X(x) \log \left(\frac{p_X(x)}{p_\theta(x)} \right) dx$$

Important Note

KL divergence is **not symmetric**:

$$D_{KL}(P_X \parallel P_\theta) \neq D_{KL}(P_\theta \parallel P_X)$$

Thus:

•

$$D_{KL}(P_X \parallel P_\theta)$$

is called the **Forward KL**

•

$$D_{KL}(P_\theta \parallel P_X)$$

is called the **Reverse KL**

Both behave differently during optimization.

b) Jensen–Shannon (JS) Divergence

Choose:

$$f(u) = \frac{1}{2} \left(u \log u - (u+1) \log \left(\frac{u+1}{2} \right) \right)$$

This produces the Jensen–Shannon divergence.

JS divergence is symmetric and is heavily used in:

- Generative Adversarial Networks (GANs)

c) Total Variation Divergence

Choose:

$$f(u) = \frac{1}{2} |u - 1|$$

Then:

$$D_f(P_X \parallel P_\theta)$$

becomes the Total Variation divergence.

It measures the maximal difference between probabilities assigned by two distributions.

Key Insight

Different choices of the function $f(u)$ lead to different divergence metrics.

Thus:

$$f(u) \implies \text{Choice of divergence}$$

and different divergences lead to different:

- Optimization behavior
- Training stability
- Sample quality
- Mode coverage

Therefore, choosing the divergence metric is one of the most important design decisions in Generative AI.

f -Divergence Minimization

Objective

Our goal is to design an algorithm that minimizes the divergence

$$D_f(P_X \parallel P_\theta)$$

without explicitly knowing either:

$$P_X \quad \text{or} \quad P_\theta$$

but having access to samples from both distributions.

Available Samples

Samples from P_X

We are given a dataset:

$$D = \{x_1, x_2, x_3, \dots, x_n\}$$

where:

$$x_i \stackrel{\text{iid}}{\sim} P_X$$

These are real samples from the unknown data distribution.

Samples from P_θ

We generate samples using a parametric model:

$$\tilde{x} = g_\theta(z)$$

where:

$$z \sim \mathcal{N}(0, I)$$

Thus:

$$\tilde{x}_i \sim P_\theta$$

These are synthetic/generated samples from the model distribution.

Key Idea

A fundamental idea in statistics and machine learning is:

Integrals involving probability density functions can be approximated using samples drawn from those distributions.

This is the mathematical foundation behind nearly all modern machine learning algorithms.

Example

Suppose we want to compute the following integral:

$$I = \int_{\mathcal{X}} h(x) p_X(x) dx$$

where:

- $h(x)$ is any arbitrary function
- $p_X(x)$ is the density function

Directly computing this integral is difficult because:

- $p_X(x)$ is unknown
- High-dimensional integration is computationally intractable

However, assume we have samples:

$$x_1, x_2, x_3, \dots, x_N \stackrel{\text{iid}}{\sim} P_X$$

Then we can estimate the integral using sample averages.

Law of the Unconscious Statistician (LOTUS)

The **Law of the Unconscious Statistician (LOTUS)** states:

$$\mathbb{E}_{P_X}[h(X)] = \int_{\mathcal{X}} h(x) p_X(x) dx$$

Thus:

$$I = \mathbb{E}_{P_X}[h(X)]$$

So the integral can be interpreted as an expectation.

Law of Large Numbers (LLN)

The **Law of Large Numbers** states:

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N h(x_i) = \mathbb{E}_{P_X}[h(X)]$$

Therefore:

$$\int_{\mathcal{X}} h(x) p_X(x) dx \approx \frac{1}{N} \sum_{i=1}^N h(x_i)$$

provided:

$$x_i \stackrel{\text{iid}}{\sim} P_X$$

and N is sufficiently large.

Interpretation

This result is extremely important:

Expectations over unknown distributions can be approximated using data samples.

Thus, we do not necessarily need to know:

$$p_X(x)$$

explicitly.

We only need samples from the distribution.

Why Large Datasets Matter

The convergence:

$$\frac{1}{N} \sum_{i=1}^N h(x_i) \rightarrow \mathbb{E}_{P_X}[h(X)]$$

happens as:

$$N \rightarrow \infty$$

Therefore:

- More data gives better approximation of the true distribution
- Larger datasets improve generalization
- High-capacity models require enormous sample sizes

This is one reason why modern AI systems such as:

- ChatGPT
- AlphaFold
- Diffusion Models

are trained on internet-scale datasets.

Core Idea Behind f -Divergence Minimization

Suppose the divergence:

$$D_f(P_X \parallel P_\theta)$$

can be rewritten in terms of expectations over:

$$P_X \quad \text{and} \quad P_\theta$$

Then:

- Expectations over P_X can be approximated using dataset samples
- Expectations over P_θ can be approximated using generated samples

Therefore:

$$D_f(P_X \parallel P_\theta)$$

can be approximately computed and optimized using samples alone.
This forms the foundation of:

- GANs
- Variational methods
- Monte Carlo estimation
- Modern generative learning algorithms

Expressing f -Divergence in Terms of Expectations

Recall the definition of f -divergence:

$$D_f(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_\theta(x) f\left(\frac{p_X(x)}{p_\theta(x)}\right) dx$$

Our objective is to rewrite this divergence in terms of expectations so that it can be estimated using samples.

Convex Conjugate Function

Suppose:

$$f(u)$$

is a convex function.

Then its **convex conjugate** (Fenchel conjugate) is defined as:

$$f^*(t) = \sup_{u \in \text{dom}(f)} \{ut - f(u)\}$$

where:

- u is the primal variable
- t is the dual variable

Aside : What is a Convex Function?

A function $f(u)$ is convex iff for all:

$$u_1, u_2 \in \mathcal{U}$$

and:

$$0 \leq \alpha_1, \alpha_2 \leq 1$$

with:

$$\alpha_1 + \alpha_2 = 1$$

the following holds:

$$\alpha_1 f(u_1) + \alpha_2 f(u_2) \geq f(\alpha_1 u_1 + \alpha_2 u_2)$$

Geometric Interpretation

The line segment joining two points on the function lies above the function itself.

Example of Convex Function

Consider:

$$f(u) = u^2$$

Take:

$$u_1 = 1, \quad u_2 = 3, \quad \alpha_1 = \alpha_2 = \frac{1}{2}$$

Then:

Left-hand side:

$$\frac{1}{2}f(1) + \frac{1}{2}f(3) = \frac{1}{2}(1) + \frac{1}{2}(9) = 5$$

Right-hand side:

$$f\left(\frac{1+3}{2}\right) = f(2) = 4$$

Thus:

$$5 \geq 4$$

Hence:

$$f(u) = u^2$$

is convex.

Properties of the Conjugate Function

i) $f^*(t)$ is also convex

If:

$$f(u) = u^2$$

then:

$$f^*(t) = \sup_u (ut - u^2)$$

Solving:

$$\frac{d}{du}(ut - u^2) = 0$$

gives:

$$t - 2u = 0$$

Thus:

$$u = \frac{t}{2}$$

Substituting back:

$$f^*(t) = \frac{t^2}{4}$$

which is also convex.

ii) Double Conjugate Property

Applying the conjugate twice recovers the original function:

$$[f^*(t)]^* = f(u)$$

Thus:

$$f(u) = \sup_{t \in \text{dom}(f^*)} \{tu - f^*(t)\}$$

This is known as the Fenchel dual representation.

Applying Conjugates to f -Divergence

Start with:

$$D_f(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_\theta(x) f\left(\frac{p_X(x)}{p_\theta(x)}\right) dx$$

Define:

$$u = \frac{p_X(x)}{p_\theta(x)}$$

Then:

$$D_f(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_\theta(x) f(u) dx$$

Using the conjugate representation:

$$f(u) = \sup_t \{tu - f^*(t)\}$$

Substitute into the divergence:

$$D_f(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_\theta(x) \sup_t \{tu - f^*(t)\} dx$$

Substituting:

$$u = \frac{p_X(x)}{p_\theta(x)}$$

gives:

$$D_f(P_X \parallel P_\theta) = \int_{\mathcal{X}} p_\theta(x) \sup_t \left\{ t \frac{p_X(x)}{p_\theta(x)} - f^*(t) \right\} dx$$

Introducing a Function $T(x)$

The optimal value of t generally depends on x .

Therefore we replace:

$$t$$

with a function:

$$T(x)$$

where:

$$T : \mathcal{X} \rightarrow \text{dom}(f^*)$$

and:

$$T(x) \in \mathcal{T}$$

with:

$$\mathcal{T}$$

being a space of functions.

Thus:

$$D_f(P_X \parallel P_\theta) = \sup_{T(x) \in \mathcal{T}} \int_{\mathcal{X}} p_\theta(x) \left[T(x) \frac{p_X(x)}{p_\theta(x)} - f^*(T(x)) \right] dx$$

In practice, the chosen function space:

$$\mathcal{T}$$

may not contain the true optimal function.

Therefore we obtain a lower bound:

$$D_f(P_X \parallel P_\theta) \geq \sup_{T(x) \in \mathcal{T}} \int_{\mathcal{X}} p_\theta(x) \left[T(x) \frac{p_X(x)}{p_\theta(x)} - f^*(T(x)) \right] dx$$

Simplifying:

$$D_f(P_X \parallel P_\theta) \geq \sup_{T(x) \in \mathcal{T}} \left[\int_{\mathcal{X}} p_X(x) T(x) dx - \int_{\mathcal{X}} p_\theta(x) f^*(T(x)) dx \right]$$

Recognizing expectations:

$$D_f(P_X \parallel P_\theta) \geq \sup_{T(x) \in \mathcal{T}} [\mathbb{E}_{P_X}[T(x)] - \mathbb{E}_{P_\theta}[f^*(T(x))]]$$

Final Variational Form

Thus, the f -divergence can be expressed variationally as:

$$D_f(P_X \parallel P_\theta) \geq \sup_{T \in \mathcal{T}} (\mathbb{E}_{P_X}[T(x)] - \mathbb{E}_{P_\theta}[f^*(T(x))])$$

Why is this Important?

This result is extremely important because:

- Expectations can be approximated using samples
- We no longer require explicit knowledge of densities
- The optimization becomes sample-based

Thus:

- Real samples approximate:

$$\mathbb{E}_{P_X}[T(x)]$$

- Generated samples approximate:

$$\mathbb{E}_{P_\theta}[f^*(T(x))]$$

This forms the mathematical foundation of:

- Variational Divergence Minimization
- GANs
- Adversarial learning
- Density-ratio estimation methods